Assignment-11.2

HT NO: 2303A51665

Name: D.Sandeep

Batch-23

Task Description #1 – Stack Implementation
Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods. Sample Input Code: class Stack:
pass
Expected Output:

- A functional stack implementation with all required methods and docstrings.

```python
#Generate a Python program that implements a Stack class using a list with push(), pop(), peek(),
# and is_empty() methods, including proper docstrings and error handling when pop or peek is called
# on an empty stack. Also include example usage at the end demonstrating the stack operations.
class Stack:
    """
    A simple implementation of a stack data structure using a list.
    """
    def __init__(self):
        """Initialize an empty stack."""
        self.items = []
    def push(self, item):
        """
        Add an item to the top of the stack.
        :param item: The item to be added to the stack.
        """
        self.items.append(item)
    def pop(self):
        """
        Remove and return the item at the top of the stack.
        :return: The item at the top of the stack.
        :raises IndexError: If the stack is empty.
        """
        if not self.is_empty():
            return self.items.pop()
        else:
            raise IndexError("Stack is empty")s
    def peek(self):
        """
        Return the item at the top of the stack without removing it.
        :return: The item at the top of the stack.
        :raises IndexError: If the stack is empty.
        """
        if not self.is_empty():
            return self.items[-1]
        else:
            raise IndexError("Stack is empty")
    def is_empty(self):
        """
        Check if the stack is empty.
        :return: True if the stack is empty, False otherwise.
        """
```

```python
        raise IndexError("Stack is empty")
    def is_empty(self):
        """
        Check if the stack is empty.
        :return: True if the stack is empty, False otherwise.
        """
        return len(self.items) == 0
# Example usage
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)
    print("Stack after pushing 1, 2, 3:", stack.items)
    print("Top item (peek):", stack.peek())
    print("Is stack empty?", stack.is_empty())
    print("Popped item:", stack.pop())
    print("Stack after popping:", stack.items)
    print("Popped item:", stack.pop())
    print("Popped item:", stack.pop())
    try:
        print("Attempting to pop from an empty stack...")
        stack.pop()
    except IndexError as e:
        print(e)
    try:
        print("Attempting to peek from an empty stack...")
        stack.peek()
    except IndexError as e:
        print(e)
```

```
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
Stack after pushing 1, 2, 3: [1, 2, 3]
Top item (peek): 3
Is stack empty? False
Popped item: 3
Stack after popping: [1, 2]
Popped item: 2
Popped item: 1
Attempting to pop from an empty stack...
Stack is empty
Attempting to peek from an empty stack...
Stack is empty
PS C:\Users\HP\Documents\AI-Lab>
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

pass

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

```python
#Generate a Python program that implements a Queue class using a list following
# FIFO (First-In-First-Out) principle with enqueue(), dequeue(), peek(), and size() methods.
# Include proper docstrings for the class and methods, handle errors when dequeue or peek is called
# on an empty queue, and show example usage demonstrating queue operations.
class Queue:
    """
    A simple implementation of a queue data structure using a list.
    """
    def __init__(self):
        """Initialize an empty queue."""
        self.items = []
    def enqueue(self, item):
        """
        Add an item to the end of the queue.
        :param item: The item to be added to the queue.
        """
        self.items.append(item)
    def dequeue(self):
        """
        Remove and return the item at the front of the queue.
        :return: The item at the front of the queue.
        :raises IndexError: If the queue is empty.
        """
        if not self.is_empty():
            return self.items.pop(0)
        else:
            raise IndexError("Queue is empty")
    def peek(self):
        """
        Return the item at the front of the queue without removing it.
        :return: The item at the front of the queue.
        :raises IndexError: If the queue is empty.
        """
        if not self.is_empty():
            return self.items[0]
        else:
            raise IndexError("Queue is empty")
    def size(self):
        """
        Return the number of items in the queue.
        :return: The size of the queue.
        """
        return len(self.items)
    def is_empty(self):
        """
        Check if the queue is empty.
        :return: True if the queue is empty, False otherwise.
        """
        return len(self.items) == 0
```

```python
        return len(self.items) == 0
# Example usage
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)
    print("Queue after enqueuing 1, 2, 3:", queue.items)
    print("Front item (peek):", queue.peek())
    print("Queue size:", queue.size())
    print("Dequeued item:", queue.dequeue())
    print("Queue after dequeuing:", queue.items)
    print("Dequeued item:", queue.dequeue())
    print("Dequeued item:", queue.dequeue())
    try:
        print("Attempting to dequeue from an empty queue...")
        queue.dequeue()
    except IndexError as e:
        print(e)
    try:
        print("Attempting to peek from an empty queue...")
        queue.peek()
    except IndexError as e:
        print(e)
```

```
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
Queue after enqueuing 1, 2, 3: [1, 2, 3]
Front item (peek): 1
Queue size: 3
Dequeued item: 1
Queue after dequeuing: [2, 3]
Queue after enqueuing 1, 2, 3: [1, 2, 3]
Front item (peek): 1
Queue size: 3
Dequeued item: 1
Queue after dequeuing: [2, 3]
Dequeued item: 2
Dequeued item: 3
Attempting to dequeue from an empty queue...
Queue is empty
Attempting to peek from an empty queue...
Queue is empty
PS C:\Users\HP\Documents\AI-Lab>
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods. Sample Input Code: class Node:

pass class

LinkedList:

pass

Expected Output:

- A working linked list implementation with clear method documentation.

```python
#Generate a Python program to implement a Singly Linked List using Node and LinkedList classes with
#  insert() and display() methods. Include proper docstrings for all classes and methods, and provide
# example usage demonstrating insertion and display of linked list elements.
class Node:
    """
    A class representing a node in a singly linked list.
    """
    def __init__(self, data):
        """
        Initialize a new node with the given data and set the next pointer to None.
        :param data: The data to be stored in the node.
        """
        self.data = data
        self.next = None
class LinkedList:
    """
    A class representing a singly linked list.
    """
    def __init__(self):
        """Initialize an empty linked list."""
        self.head = None
    def insert(self, data):
        """
        Insert a new node with the given data at the end of the linked list.
        :param data: The data to be stored in the new node.
        """
        new_node = Node(data)
        if not self.head:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node
    def display(self):
        """
        Display the elements of the linked list.
        :return: A list containing the elements of the linked list.
        """
        elements = []
        current = self.head
        while current:
            elements.append(current.data)
            current = current.next
        return elements
# Example usage
if __name__ == "__main__":
    linked_list = LinkedList()
    linked_list.insert(1)
    linked_list.insert(2)
    linked_list.insert(3)
    print("Linked List elements:", linked_list.display())
```

```
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
Linked List elements: [1, 2, 3]
PS C:\Users\HP\Documents\AI-Lab>
```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

class BST:

pass

Expected Output:

- BST implementation with recursive insert and traversal methods.

C:\Users\HP\Documents\AI-Lab\rectangle_refactored.py

```python
#Generate a [illegible] Search Tree (BST) using Node and BST classes with a
# recursive insert() method and an in-order traversal method. Include proper docstrings for all
# classes and methods, and provide example usage demonstrating insertion and traversal of the BST.
class Node:
    """
    A class representing a node in a binary search tree.
    """
    def __init__(self, data):
        """
        Initialize a new node with the given data and set left and right pointers to None.
        :param data: The data to be stored in the node.
        """
        self.data = data
        self.left = None
        self.right = None
class BST:
    """
    A class representing a binary search tree (BST).
    """
    def __init__(self):
        """Initialize an empty binary search tree."""
        self.root = None
    def insert(self, data):
        """
        Insert a new node with the given data into the BST.
        :param data: The data to be stored in the new node.
        """
        if self.root is None:
            self.root = Node(data)
        else:
            self._insert_recursive(self.root, data)
    def _insert_recursive(self, current_node, data):
        """
        Helper method to recursively insert a new node into the BST.
        :param current_node: The current node being compared.
        :param data: The data to be inserted.
        """
        if data < current_node.data:
            if current_node.left is None:
                current_node.left = Node(data)
            else:
                self._insert_recursive(current_node.left, data)
        else:
            if current_node.right is None:
                current_node.right = Node(data)
            else:
                self._insert_recursive(current_node.right, data)
    def in_order_traversal(self):
        """
        Perform an in-order traversal of the BST and return a list of elements in sorted order.
        :return: A list containing the elements of the BST in sorted order.
        """
```

```python
            self._insert_recursive(current_node.right, data)
    def in_order_traversal(self):
        """
        Perform an in-order traversal of the BST and return a list of elements in sorted order.
        :return: A list containing the elements of the BST in sorted order.
        """
        elements = []
        self._in_order_recursive(self.root, elements)
        return elements
    def _in_order_recursive(self, current_node, elements):
        """
        Helper method to recursively perform an in-order traversal of the BST.
        :param current_node: The current node being visited.
        :param elements: The list to store the elements of the BST.
        """
        if current_node:
            self._in_order_recursive(current_node.left, elements)
            elements.append(current_node.data)
            self._in_order_recursive(current_node.right, elements)
# Example usage
if __name__ == "__main__":
    bst = BST()
    bst.insert(5)
    bst.insert(3)
    bst.insert(7)
    bst.insert(2)
    bst.insert(4)
    bst.insert(6)
    bst.insert(8)
    print("In-order traversal of the BST:", bst.in_order_traversal())
```

```
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
In-order traversal of the BST: [2, 3, 4, 5, 6, 7, 8]
PS C:\Users\HP\Documents\AI-Lab>
```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods. Sample Input Code: class HashTable:

pass

Expected Output:

- Collision handling using chaining, with well-commented methods.

```python
#Generate a Python program to implement a HashTable class using a list of buckets with collision handling
# through chaining.Include insert, search, and delete methods with proper docstrings
# or comments, and provide example usage demonstrating those operations.
class HashTable:
    """
    A simple implementation of a hash table using chaining for collision handling.
    """
    def __init__(self, size=10):
        """
        Initialize the hash table with a specified size and create empty buckets.
        :param size: The size of the hash table (number of buckets).
        """
        self.size = size
        self.table = [[] for _ in range(size)]
    def _hash(self, key):
        """
        Compute the hash value for a given key.
        :param key: The key to be hashed.
        :return: The hash value corresponding to the key.
        """
        return hash(key) % self.size
    def insert(self, key, value):
        """
        Insert a key-value pair into the hash table. If the key already exists, update its value.
        :param key: The key to be inserted.
        :param value: The value associated with the key.
        """
        index = self._hash(key)
        bucket = self.table[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)  # Update existing key
                return
        bucket.append((key, value))  # Insert new key-value pair
    def search(self, key):
        """
        Search for a value associated with a given key in the hash table.
        :param key: The key to be searched.
        :return: The value associated with the key if found, else None.
        """
        index = self._hash(key)
        bucket = self.table[index]
        for k, v in bucket:
            if k == key:
                return v
        return None  # Key not found
    def delete(self, key):
        """
```

```python
        return None  # Key not found
    def delete(self, key):
        """
        Delete a key-value pair from the hash table based on the given key.
        :param key: The key to be deleted.
        :raises KeyError: If the key is not found in the hash table.
        """
        index = self._hash(key)
        bucket = self.table[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                del bucket[i]  # Remove the key-value pair
                return
        raise KeyError("Key not found")
# Example usage
if __name__ == "__main__":
    hash_table = HashTable()
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    hash_table.insert("city", "New York")
    print("Search for 'name':", hash_table.search("name"))
    print("Search for 'age':", hash_table.search("age"))
    print("Search for 'city':", hash_table.search("city"))
    print("Search for 'country' (not found):", hash_table.search("country"))
    hash_table.delete("age")
    print("Search for 'age' after deletion:", hash_table.search("age"))
    try:
        hash_table.delete("age")  # Attempt to delete a non-existent key
    except KeyError as e:
        print(e)
```

```
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
Search for 'name': Alice
Search for 'age': 30
Search for 'name': Alice
Search for 'age': 30
Search for 'city': New York
Search for 'city': New York
Search for 'country' (not found): None
Search for 'age' after deletion: None
'Key not found'
'Key not found'
'Key not found'
PS C:\Users\HP\Documents\AI-Lab>

'Key not found'
PS C:\Users\HP\Documents\AI-Lab>

'Key not found'
PS C:\Users\HP\Documents\AI-Lab>
```

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

pass

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

```python
#Write a Python program to implement a Graph using an adjacency list. Include methods for
# adding vertices, adding edges, and displaying the graph. Provide example usage demonstrating these operations.
class Graph:
    """
    A simple implementation of a graph using an adjacency list.
    """
    def __init__(self):
        """Initialize an empty graph."""
        self.graph = {}
    def add_vertex(self, vertex):
        """
        Add a vertex to the graph.
        :param vertex: The vertex to be added.
        """
        if vertex not in self.graph:
            self.graph[vertex] = []
    def add_edge(self, vertex1, vertex2):
        """
        Add an edge between two vertices in the graph. If the vertices do not exist, they will be added.
        :param vertex1: The first vertex of the edge.
        :param vertex2: The second vertex of the edge.
        """
        self.add_vertex(vertex1)
        self.add_vertex(vertex2)
        self.graph[vertex1].append(vertex2)
        self.graph[vertex2].append(vertex1)  # For undirected graph
    def display(self):
        """Display the adjacency list representation of the graph."""
        for vertex, edges in self.graph.items():
            print(f"{vertex}: {edges}")
# Example usage
if __name__ == "__main__":
    graph = Graph()
    graph.add_edge("A", "B")
    graph.add_edge("A", "C")
    graph.add_edge("B", "D")
    graph.add_edge("C", "D")
    graph.add_edge("D", "E")
    print("Graph adjacency list:")
    graph.display()
```

```
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
Graph adjacency list:
A: ['B', 'C']
B: ['A', 'D']
C: ['A', 'D']
D: ['B', 'C', 'E']
E: ['D']
PS C:\Users\HP\Documents\AI-Lab>
```

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module. Sample Input Code: class PriorityQueue:

pass

Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority), and display methods.

```python
#Write a Python program to implement a Priority Queue using Python's heapq module.
# Include methods for adding items with with enqueue (priority), dequeue (highest priority), and display methods
import heapq
class PriorityQueue:
    def __init__(self):
        self._queue = []
        self._index = 0

    def enqueue(self, item, priority):
        heapq.heappush(self._queue, (priority, self._index, item))
        self._index += 1

    def dequeue(self):
        if not self._queue:
            raise IndexError("dequeue from an empty priority queue")
        return heapq.heappop(self._queue)[-1]

    def display(self):
        for priority, index, item in self._queue:
            print(f"Item: {item}, Priority: {priority}")
# Example usage
if __name__ == "__main__":
    pq = PriorityQueue()
    pq.enqueue("task1", priority=3)
    pq.enqueue("task2", priority=1)
    pq.enqueue("task3", priority=2)
    print("Priority Queue:")
    pq.display()
    print("\nDequeueing items:")
    while True:
        try:
            print(pq.dequeue())
        except IndexError:
            print("Priority Queue is empty.")
            break
```

```
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
Priority Queue:
Item: task2, Priority: 1
Item: task1, Priority: 3
Item: task3, Priority: 2

Dequeueing items:
Dequeueing items:
task2
task2
task3
task1
Priority Queue is empty.
PS C:\Users\HP\Documents\AI-Lab>
```

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using collections.deque. Sample Input Code:

class DequeDS:

pass

Expected Output:

• Insert and remove from both ends with docstrings.

```python
#Write a Python program to implement a double-ended queue (Deque) using collections.deque.
# Include methods for adding and removing items from both ends with docstrings
from collections import deque
class Deque:
    def __init__(self):
        self._deque = deque()

    def append(self, item):
        """Add an item to the right end of the deque."""
        self._deque.append(item)

    def appendleft(self, item):
        """Add an item to the left end of the deque."""
        self._deque.appendleft(item)

    def pop(self):
        """Remove and return an item from the right end of the deque."""
        if not self._deque:
            raise IndexError("pop from an empty deque")
        return self._deque.pop()

    def popleft(self):
        """Remove and return an item from the left end of the deque."""
        if not self._deque:
            raise IndexError("popleft from an empty deque")
        return self._deque.popleft()

    def display(self):
        """Display the current state of the deque."""
        print(list(self._deque))
# Example usage
if __name__ == "__main__":
    dq = Deque()
    dq.append("item1")
    dq.appendleft("item2")
    dq.append("item3")
    print("Deque:")
    dq.display()
    print("\nPopping items:")
    print(dq.pop())
    print(dq.popleft())
    print("Deque after popping:")
    dq.display()
```

```
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
Deque:
['item2', 'item1', 'item3']

Popping items:
item3
item2
Deque after popping:
['item1']
PS C:\Users\HP\Documents\AI-Lab>
```

Task Description #9 Real-Time Application Challenge –
Choose the Right Data Structure Scenario:
Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.

4. Bus Scheduling System – Maintain bus routes and stop connections.

5. Cafeteria Order Queue – Serve students in the order they arrive. Student Task:

- For each feature, select the most appropriate data structure from the list below: o Stack o Queue o Priority Queue o Linked List o Binary Search Tree (BST) o Graph o Hash Table o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification. • A functional Python program implementing the chosen feature with comments and docstrings.

```python
#generate a code for the Campus Resource Management System features and select the most suitable data structure from the following:
# Stack, Queue, Priority Queue, Linked List, Binary Search Tree (BST), Graph, Hash Table,
#  and Deque.Create a table mapping Feature → Chosen Data Structure → Justification and explain the reason for each selection in 2-3 sentences.
#Choose one feature from the Campus Resource Management System and implement it as a Python program using t
# he most appropriate data structure.Include comments, docstrings, and example usage demonstrating how the system works.
from collections import deque
class CafeteriaQueue:
    """
    A queue system for managing cafeteria orders.
    Students are served in FIFO (First-In-First-Out) order.
    """
    def __init__(self):
        """Initialize an empty queue."""
        self.orders = deque()

    def add_student(self, name):
        """
        Add a student to the order queue.

        Parameters:
        name (str): Name of the student placing an order.
        """
        self.orders.append(name)
        print(name, "added to the queue.")

    def serve_student(self):
        """
        Serve the next student in the queue.
        """
        if not self.orders:
            print("No students in queue.")
            return
        student = self.orders.popleft()
        print("Serving:", student)

    def display_queue(self):
        """
        Display the current queue of students.
        """
        print("Current Queue:", list(self.orders))
# Example Usage
queue = CafeteriaQueue()
queue.add_student("Ravi")
queue.add_student("Anita")
queue.add_student("Kiran")
queue.display_queue()
queue.serve_student()
queue.display_queue()
```

```
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
Ravi added to the queue.
Anita added to the queue.
Kiran added to the queue.
Current Queue: ['Ravi', 'Anita', 'Kiran']
Serving: Ravi
Current Queue: ['Anita', 'Kiran']
PS C:\Users\HP\Documents\AI-Lab>
```

Task Description #10: Smart Hospital Management System –
Data
Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using

ID.

4. Doctor Appointment Scheduling – Appointments sorted by time. 5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack o Queue o Priority Queue o Linked List o Binary Search Tree (BST) o Graph o Hash Table o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification. • A functional Python program implementing the chosen feature with comments and docstrings.

```python
#write a code for the Smart Hospital Management System features and select the most suitable data structure from:
# Stack, Queue, Priority Queue, Linked List, Binary Search Tree (BST), Graph, Hash Table,
# and Deque.Create a table mapping Feature → Chosen Data Structure → Justification and explain each choice in 2-3 sentences.
#Choose one feature from the Smart Hospital Management System and implement it as a Python program using the most
# appropriate data structure.Include comments, docstrings, and example usage to demonstrate how the system works.
import heapq
class EmergencyRoom:
    """
    A priority queue system for managing patients in an emergency room.
    Patients are served based on the severity of their condition (higher priority first).
    """
    def __init__(self):
        """Initialize an empty priority queue."""
        self.patients = []
        self.index = 0

    def add_patient(self, name, severity):
        """
        Add a patient to the emergency room queue.

        Parameters:
        name (str): Name of the patient.
        severity (int): Severity level of the patient's condition (higher is more severe).
        """
        heapq.heappush(self.patients, (-severity, self.index, name))
        self.index += 1
        print(name, "added to the emergency room with severity", severity)

    def serve_patient(self):
        """
        Serve the next patient in the emergency room based on severity.
        """
        if not self.patients:
            print("No patients in the emergency room.")
            return
        patient = heapq.heappop(self.patients)[-1]
        print("Serving:", patient)
```

```python
    def display_patients(self):
        """
        Display the current list of patients in the emergency room.
        """
        print("Current Patients in Emergency Room:")
        for severity, index, name in sorted(self.patients, reverse=True):
            print(f"Patient: {name}, Severity: {-severity}")
# Example Usage
er = EmergencyRoom()
er.add_patient("John Doe", severity=5)
er.add_patient("Jane Smith", severity=8)
er.add_patient("Emily Davis", severity=3)
er.display_patients()
```

```
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
John Doe added to the emergency room with severity 5
Jane Smith added to the emergency room with severity 8
Emily Davis added to the emergency room with severity 3
Current Patients in Emergency Room:
Patient: Emily Davis, Severity: 3
Patient: John Doe, Severity: 5
Patient: Jane Smith, Severity: 8
PS C:\Users\HP\Documents\AI-Lab>
```

Task Description #11: Smart City Traffic Control System A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.

Student Task

• For each feature, select the most appropriate data structure from the list below: o StackQueue o Priority Queue o Linked 16
List o Binary Search Tree (BST) o Graph 16
Hash TableDeque 16

• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification. • A functional Python program implementing the chosen feature with comments and docstrings.

```python
#write a code for A city plans a Smart Traffic Management System that includes:
#Select the most appropriate data structure (Stack, Queue, Priority Queue, Linked List, BST, Graph, Hash Table, Deque)
# for each Smart City Traffic Control System feature and justify the choice in 2-3 sentences.
#A table mapping feature + chosen data structure + justification. A functional Python program implementing the chosen feature with comments and docstrings.
from collections import deque

class TrafficSignalQueue:
    """
    A queue system to manage vehicles waiting at a traffic signal.
    Vehicles pass the signal in FIFO order.
    """

    def __init__(self):
        """Initialize an empty vehicle queue."""
        self.queue = deque()

    def add_vehicle(self, vehicle):
        """
        Add a vehicle to the traffic signal queue.

        Parameters:
        vehicle (str): Vehicle number or name.
        """
        self.queue.append(vehicle)
        print(vehicle, "arrived at the signal.")

    def pass_signal(self):
        """
        Allow the first vehicle in the queue to pass the signal.
        """
        if not self.queue:
            print("No vehicles waiting.")
            return
        vehicle = self.queue.popleft()
        print(vehicle, "passed the signal.")
    def display_queue(self):
        """
        Display all vehicles currently waiting.
        """
        print("Vehicles waiting:", list(self.queue))
# Example Usage
traffic = TrafficSignalQueue()
traffic.add_vehicle("Car1")
traffic.add_vehicle("Bike2")
traffic.add_vehicle("Bus3")
traffic.display_queue()
traffic.pass_signal()
traffic.display_queue()
```

```
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
Car1 arrived at the signal.
Bike2 arrived at the signal.
Bus3 arrived at the signal.
Vehicles waiting: ['Car1', 'Bike2', 'Bus3']
Car1 passed the signal.
Vehicles waiting: ['Bike2', 'Bus3']
PS C:\Users\HP\Documents\AI-Lab>
```

Task Description #12: Smart E-Commerce Platform – Data Structure

Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.

4. Product Search Engine – Fast lookup of products using product ID. 5. Delivery Route Planning – Connect warehouses and delivery locations. Student Task

- For each feature, select the most appropriate data structure from the list below: o Stack o Queue o Priority Queue o Linked

List o Binary Search

Tree (BST) o Graph o

Hash Table o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```python
#generate a code for  the Smart E-Commerce Platform features, choose the most suitable data structure
# (Stack, Queue, Priority Queue, Linked List, BST, Graph, Hash Table, Deque) for each,
# justify the choice in 2-3 sentences, and implement one feature as a Python program with comments and docstrings.
class ShoppingCart:
    """
    A class representing a shopping cart in an e-commerce platform.
    The cart allows adding and removing items, and viewing the current items in the cart.
    """
    def __init__(self):
        """Initialize an empty shopping cart."""
        self.cart = []
    def add_item(self, item):
        """
        Add an item to the shopping cart.

        Parameters:
        item (str): The name of the item to be added to the cart.
        """
        self.cart.append(item)
        print(item, "added to the cart.")
    def remove_item(self, item):
        """
        Remove an item from the shopping cart.

        Parameters:
        item (str): The name of the item to be removed from the cart.
        """
        if item in self.cart:
            self.cart.remove(item)
            print(item, "removed from the cart.")
        else:
            print(item, "not found in the cart.")
    def view_cart(self):
        """Display the current items in the shopping cart."""
        print("Current items in the cart:", self.cart)
# Example Usage
cart = ShoppingCart()
cart.add_item("Laptop")
cart.add_item("Headphones")
cart.view_cart()
cart.remove_item("Headphones")
cart.view_cart()
cart.remove_item("Smartphone")
```

```
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
Laptop added to the cart.
Headphones added to the cart.
Current items in the cart: ['Laptop', 'Headphones']
Headphones removed from the cart.
Current items in the cart: ['Laptop']
Smartphone not found in the cart.
PS C:\Users\HP\Documents\AI-Lab>
```

Task #13: Smart Logistics & Warehouse Management Scenario:

A logistics company wants to implement:

1. Shipment Processing
2. Urgent Shipment Priority
3. Product Lookup by Code
4. Warehouse Network Connectivity
5. Inventory Add/Remove Operations

Student Task

• For each feature, select the most appropriate data structure from the list below: o Stack o Queue o Priority Queue o Linked List o Binary Search Tree (BST) o Graph o

Hash Table o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification. • A functional Python program implementing the chosen feature with comments and docstrings.

```python
#write a code for the Smart Logistics & Warehouse Management features and select the most suitable data structure (Stack, Queue, Priority Queue, Linked List, BST, Graph, Hash Table, Deque)
# for each feature with a 2-3 sentence justification, then implement one feature as a Python program with comments, docstrings, and example usage.
class WarehouseInventory:
    """
    A class representing the inventory management system of a warehouse.
    It allows adding, removing, and viewing items in the inventory.
    """
    def __init__(self):
        """Initialize an empty inventory."""
        self.inventory = {}
    def add_item(self, item, quantity):
        """
        Add an item to the inventory.
        Parameters:
        item (str): The name of the item to be added.
        quantity (int): The quantity of the item to be added.
        """
        if item in self.inventory:
            self.inventory[item] += quantity
        else:
            self.inventory[item] = quantity
        print(f"{quantity} units of {item} added to inventory.")
    def remove_item(self, item, quantity):
        """
        Remove an item from the inventory.
        Parameters:
        item (str): The name of the item to be removed.
        quantity (int): The quantity of the item to be removed.
        """
        if item in self.inventory and self.inventory[item] >= quantity:
            self.inventory[item] -= quantity
            print(f"{quantity} units of {item} removed from inventory.")
            if self.inventory[item] == 0:
                del self.inventory[item]
                print(f"{item} is now out of stock.")
        else:
            print(f"Cannot remove {quantity} units of {item}. Not enough stock or item not found.")
    def view_inventory(self):
        """Display the current inventory."""
        print("Current Inventory:")
        for item, quantity in self.inventory.items():
            print(f"{item}: {quantity} units")
# Example Usage
warehouse = WarehouseInventory()
warehouse.add_item("Widget A", 100)
warehouse.add_item("Widget B", 50)
warehouse.view_inventory()
warehouse.remove_item("Widget A", 20)
warehouse.view_inventory()
warehouse.remove_item("Widget A", 80)
warehouse.view_inventory()
warehouse.remove_item("Widget B", 60)
```

```
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
100 units of Widget A added to inventory.
50 units of Widget B added to inventory.
Current Inventory:
Widget A: 100 units
Widget B: 50 units
20 units of Widget A removed from inventory.
Widget A: 100 units
Widget B: 50 units
20 units of Widget A removed from inventory.
Widget B: 50 units
20 units of Widget A removed from inventory.
20 units of Widget A removed from inventory.
Current Inventory:
Widget A: 80 units
Widget B: 50 units
80 units of Widget A removed from inventory.
Widget A is now out of stock.
Current Inventory:
Widget B: 50 units
Current Inventory:
Widget B: 50 units
Cannot remove 60 units of Widget B. Not enough stock or item not found.
Cannot remove 60 units of Widget B. Not enough stock or item not found.
PS C:\Users\HP\Documents\AI-Lab>
```

Task Description #14: Smart Banking Management System – Data
Structure Challenge

A bank plans to develop a Smart Banking System that includes:

1. Customer Service Token System – Customers are served in order of arrival.
2. Loan Approval Priority Handling – High-value or urgent loans processed first.

3. Account Information Lookup – Fast retrieval of customer details using Account Number.
4. Transaction History Management – Maintain chronological transaction records.
5. ATM Network Connectivity – Represent connections between ATMs across cities.

Student Task:

• For each feature, select the most appropriate data structure from the list below: o Stack o Queue o Priority Queue o Linked List o Binary Search Tree (BST) o Graph o Hash Table o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification. • A functional Python program implementing the chosen feature with comments and docstrings.

```python
#write a code for the Smart Banking Management System features and choose the most suitable data structure (Stack, Queue, Priority Queue, Linked List,
#BST, Graph, Hash Table, Deque) for each feature with a 2-3 sentence justification, then implement one selected feature as a Python program with comments, docstrings, and example usage.
from collections import deque

class BankQueue:
    """
    A queue system for managing bank customer service tokens.
    Customers are served in First-In-First-Out (FIFO) order.
    """
    def __init__(self):
        """Initialize an empty customer queue."""
        self.queue = deque()
    def add_customer(self, name):
        """
        Add a customer to the service queue.
        Parameters:
        name (str): Customer name.
        """
        self.queue.append(name)
        print(name, "joined the service queue.")
    def serve_customer(self):
        """
        Serve the next customer in the queue.
        """
        if not self.queue:
            print("No customers waiting.")
            return
        customer = self.queue.popleft()
        print("Serving customer:", customer)
    def display_queue(self):
        """
        Display all customers currently waiting.
        """
        print("Customers waiting:", list(self.queue))
# Example Usage
bank = BankQueue()
bank.add_customer("Ravi")
bank.add_customer("Anita")
bank.add_customer("Kiran")
bank.display_queue()
bank.serve_customer()
bank.display_queue()
```

```
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
Ravi joined the service queue.
Anita joined the service queue.
Kiran joined the service queue.
Customers waiting: ['Ravi', 'Anita', 'Kiran']
Serving customer: Ravi
Customers waiting: ['Ravi', 'Anita', 'Kiran']
Serving customer: Ravi
Serving customer: Ravi
Customers waiting: ['Anita', 'Kiran']
PS C:\Users\HP\Documents\AI-Lab>
```

Task Description #15: Online Learning Management System –
Data
Structure Selection
An online education platform wants to develop a Learning Management

System that handles:

1. Student Login Authentication – Quick validation of user credentials.
2. Assignment Submission Queue – Submissions processed in order of upload.
3. Top Performer Ranking – Rank students based on scores.
4. Course Prerequisite Structure – Represent subject dependencies.
5. Undo Feature in Online Editor – Reverse recent actions.

Student Task:

- Select the most appropriate data structure for each feature from the given list.
- Justify your selection in 2–3 sentences per feature.
- Implement one selected feature in Python using AI-assisted code generation.

Expected Output:

- Feature → Data Structure → Justification table.
- Functional Python program with proper documentation.

```python
#generate a code for the Online Learning Management System features and choose the most appropriate data structure (Stack, Queue, Priority Queue, Linked List,
#BST, Graph, Hash Table, Deque)for each feature with a 2-3 sentence justification, then implement one selected feature in Python with comments, docstrings, and example usage.
class CourseEnrollment:
    """
    A class representing the course enrollment system of an online learning management platform.
    It allows students to enroll in courses and view their enrolled courses.
    """
    def __init__(self):
        """Initialize an empty enrollment system."""
        self.enrollments = {}
    def enroll_student(self, student_name, course_name):
        """
        Enroll a student in a course.
        Parameters:
        student_name (str): The name of the student enrolling in the course.
        course_name (str): The name of the course to enroll in.
        """
        if student_name not in self.enrollments:
            self.enrollments[student_name] = []
        self.enrollments[student_name].append(course_name)
        print(f"{student_name} enrolled in {course_name}.")
    def view_enrollments(self, student_name):
        """
        View the courses a student is enrolled in.
        Parameters:
        student_name (str): The name of the student whose enrollments are to be viewed.
        """
        if student_name in self.enrollments:
            print(f"{student_name} is enrolled in: {', '.join(self.enrollments[student_name])}")
        else:
            print(f"{student_name} is not enrolled in any courses.")
# Example Usage
enrollment_system = CourseEnrollment()
enrollment_system.enroll_student("Ravi", "Python Programming")
enrollment_system.enroll_student("Ravi", "Data Structures")
enrollment_system.enroll_student("Anita", "Machine Learning")
enrollment_system.view_enrollments("Ravi")
enrollment_system.view_enrollments("Anita")
enrollment_system.view_enrollments("Kiran")
```

```
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
Ravi enrolled in Python Programming.
Ravi enrolled in Data Structures.
Anita enrolled in Machine Learning.
Ravi is enrolled in: Python Programming, Data Structures
Anita is enrolled in: Machine Learning
Kiran is not enrolled in any courses.
PS C:\Users\HP\Documents\AI-Lab>
```

Task Description #16: Smart Airport Management System – Data
Structure Challenge

An airport authority plans to implement:

1. Passenger Check-In Queue – Process passengers in order of arrival.
2. Emergency Landing Priority – Aircraft in emergency get priority.
3. Flight Information Lookup – Fast search using Flight ID.
4. Air Route Connectivity – Airports connected via routes.
5. Boarding Process (Last-In First-Out Cabin Storage) – Overhead luggage management simulation.

Student Task:

- Choose suitable data structures from the provided list.
- Provide 2–3 sentence justification per feature.
- Implement one feature in Python with comments and docstrings using AI assistance.

Expected Output:

• Mapping table.

• Working Python implementation.

```python
#generate a code for the Smart Airport Management System features and select the most suitable data structure (Stack, Queue, Priority Queue, Linked List, BST, Graph,
#Hash Table, Deque) for each feature with a 2-3 sentence justification, then implement one selected feature in Python with comments, docstrings, and example usage

class FlightQueue:
    """
    A queue system for managing flight boarding at an airport.
    Passengers are boarded in First-In-First-Out (FIFO) order.
    """
    def __init__(self):
        """Initialize an empty boarding queue."""
        self.queue = deque()

    def add_passenger(self, name):
        """
        Add a passenger to the boarding queue.
        Parameters:
        name (str): Passenger name.
        """
        self.queue.append(name)
        print(name, "added to the boarding queue.")

    def board_passenger(self):
        """
        Board the next passenger in the queue.
        """
        if not self.queue:
            print("No passengers waiting to board.")
            return
        passenger = self.queue.popleft()
        print("Boarding passenger:", passenger)

    def display_queue(self):
        """
        Display all passengers currently waiting to board.
        """
        print("Passengers waiting to board:", list(self.queue))
# Example Usage
flight = FlightQueue()
flight.add_passenger("Ravi")
flight.add_passenger("Anita")
flight.add_passenger("Kiran")
flight.display_queue()
flight.board_passenger()
flight.display_queue()
```

```
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
Ravi added to the boarding queue.
Anita added to the boarding queue.
Kiran added to the boarding queue.
Anita added to the boarding queue.
Kiran added to the boarding queue.
Passengers waiting to board: ['Ravi', 'Anita', 'Kiran']
Boarding passenger: Ravi
Passengers waiting to board: ['Anita', 'Kiran']
PS C:\Users\HP\Documents\AI-Lab>
```

Task Description #17: Smart Social Media Platform – Data Structure
Selection

A social media company wants to develop a system that includes:

1. News Feed Generation – Display posts chronologically.
2. Trending Hashtags Tracker – Rank hashtags by usage frequency.
3. User Profile Search – Quick lookup using username.
4. Friend Network Representation – Represent user connections. 5. Message Undo Feature – Allow last sent message to be withdrawn.

Student Task:

• Select appropriate data structures from the list.

• Justify your choices clearly.

• Implement one selected feature as a Python program using AI- assisted code generation.

Expected Output:

• Table mapping features to data structures with justification.

• Working Python code with docstrings and comments.

```python
#generate a code for the Smart Social Media Platform features and choose the most suitable data structure (Stack, Queue, Priority Queue, Linked List, Binary Search Tree, Graph,
#Hash Table, Deque) for each feature with a clear 2-3 sentence justification. Then implement one selected feature as a Python program with proper comments, docstrings, and example usage.
class PostFeed:
    """
    A class representing the post feed of a social media platform.
    It allows users to add posts and view the feed in reverse chronological order.
    """
    def __init__(self):
        """Initialize an empty post feed."""
        self.posts = []
    def add_post(self, post):
        """
        Add a post to the feed.
        Parameters:
        post (str): The content of the post to be added.
        """
        self.posts.append(post)
        print("Post added to the feed.")
    def view_feed(self):
        """Display the posts in reverse chronological order."""
        print("Post Feed:")
        for post in reversed(self.posts):
            print(post)
# Example Usage
feed = PostFeed()
feed.add_post("Hello, world!")
feed.add_post("Welcome to my social media feed.")
feed.add_post("Enjoying the platform!")
feed.view_feed()
```

```
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
Post added to the feed.
Post added to the feed.
Post added to the feed.
Post Feed:
Enjoying the platform!
Welcome to my social media feed.
Hello, world!
PS C:\Users\HP\Documents\AI-Lab>
```

Task #18: Smart University Hostel Management

Scenario:

A university wants to build a hostel management system that manages:

1. Room Allotment
2. Medical Emergency Handling
3. Student Record Retrieval
4. Hostel Block Connectivity
5. Visitor Entry Log

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack o Queue o Priority Queue o Linked List o Binary Search Tree (BST) o Graph o

Hash Table o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification. • A functional Python program implementing the chosen feature with comments and docstrings.

```python
#write a code for  the Smart University Hostel Management System features and select the most suitable data structure (Stack, Queue, Priority Queue,
#linked list, Binary Search Tree, Graph, Hash Table, Deque) for each feature with a clear 2-3 sentence justification.Then implement one
# selected feature as a Python program with proper comments, docstrings, and example usage demonstrating how it works.
class RoomAllocation:
    """
    A class representing the room allocation system of a university hostel.
    It allows students to request rooms and view their allocated rooms.
    """
    def __init__(self):
        """Initialize an empty room allocation system."""
        self.allocations = {}
    def request_room(self, student_name, room_number):
        """
        Request a room for a student.
        Parameters:
        student_name (str): The name of the student requesting a room.
        room_number (str): The number of the room being requested.
        """
        self.allocations[student_name] = room_number
        print(f"{student_name} has been allocated room {room_number}.")
    def view_allocations(self):
        """Display all current room allocations."""
        print("Current Room Allocations:")
        for student, room in self.allocations.items():
            print(f"{student}: Room {room}")
# Example Usage
hostel = RoomAllocation()
hostel.request_room("Ravi", "101")
hostel.request_room("Anita", "102")
hostel.request_room("Kiran", "103")
hostel.view_allocations()
```

```
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
Ravi has been allocated room 101.
Anita has been allocated room 102.
Kiran has been allocated room 103.
Current Room Allocations:
Ravi: Room 101
Anita: Room 102
Kiran: Room 103
PS C:\Users\HP\Documents\AI-Lab>
```

Task #19: Smart Electricity Distribution System Scenario:

An electricity board plans to develop a system that includes:

1. Complaint Handling
2. Fault Priority Handling
3. Consumer Record Lookup
4. Power Grid Connectivity
5. Meter Reading History Tracking

Student Task

- For each feature, select the most appropriate data structure from the list below:

o Stack o Queue

o Priority
Queue o Linked
List o Binary
Search Tree
(BST) o Graph o
Hash Table o
Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification. • A functional Python program implementing the chosen feature with comments and docstrings.

```
Generate a code [redacted] the best electricity distribution system features and select the most suitable data structure (Stack, Queue, Priority Queue, Linked List, Binary Search Tree (BST),
Graph, Hash Table, Deque) [redacted] each feature with a clear 2–3 sentence justification. Then implement one selected feature as a Python program with proper comments, docstrings, and example usage demonstrating how the system works.
class PowerOutageQueue:
    """
    A queue system [redacted] managing power outage reports in an electricity distribution system.
    Outages are handled in First-In-First-Out (FIFO) order to ensure fairness.
    """
    def __init__(self):
        """Initialize an empty outage report queue."""
        self.queue = deque()

    def report_outage(self, location):
        """
        Report a power outage at a specific location.
        Parameters:
        location (str): The location where the power outage is occurring.
        """
        self.queue.append(location)
        print(f"Power outage reported at {location}.")

    def resolve_outage(self):
        """
        Resolve the next power outage in the queue.
        """
        if not self.queue:
            print("No power outages to resolve.")
            return
        location = self.queue.popleft()
        print(f"Resolving power outage at {location}.")

    def display_outages(self):
        """Display all current power outage reports."""
        print("Current Power Outage Reports:", list(self.queue))
# Example Usage
outage_system = PowerOutageQueue()
outage_system.report_outage("Sector 1")
outage_system.report_outage("Sector 2")
outage_system.report_outage("Sector 3")
outage_system.display_outages()
outage_system.resolve_outage()
outage_system.display_outages()
```

```
PS C:\Users\HP\Documents\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/Documents/AI-Lab/rectangle_refactored.py
Power outage reported at Sector 1.
Power outage reported at Sector 2.
Power outage reported at Sector 3.
Current Power Outage Reports: ['Sector 1', 'Sector 2', 'Sector 3']
Resolving power outage at Sector 1.
Current Power Outage Reports: ['Sector 2', 'Sector 3']
PS C:\Users\HP\Documents\AI-Lab>
```